

Documentação do projeto

Simulador do Modelo OSI · Comunicação de Dados · Prof. Vinícius S. Borges · 2º semestre de 2026

Esta documentação descreve o funcionamento interno do simulador: a separação entre as camadas, a estrutura dos dados, a interface de cada módulo, os parâmetros e as convenções que determinam os números produzidos. Ela serve de referência para quem precisa entender, modificar ou estender o programa. A operação do programa está nos tutoriais de execução e de uso.

1. Visão geral

O simulador reproduz, passo a passo, o percurso de uma mensagem entre dois computadores numa rede com vários roteadores. O modelo implementado é o **modelo OSI de sete camadas**: cada computador tem as sete camadas, e cada roteador tem as três inferiores (física, enlace e rede). A exibição pode reagrupar as camadas no modelo TCP/IP, mas a simulação é sempre feita nas sete camadas OSI.

Cada ação de uma camada, como gerar a mensagem, cifrar, abrir a sessão, segmentar, encapsular, rotear, enquadrar ou transmitir, produz um **evento**: um retrato imutável de quem agiu, em que camada, o que fez e como ficou a unidade de dados. A simulação de um caso roda inteira, em memória e num único processo, antes de qualquer desenho. A interface apenas percorre a lista de eventos pronta.

A rede, os endereços, os custos e os casos de teste são lidos de um arquivo editável, o `topologia.json`. A topologia de referência tem três redes locais (A, B e C), cinco computadores (H1 a H5) e quatro roteadores (R1 a R4). Não há tráfego real: nenhum socket é aberto e nenhuma thread é criada. Os cabeçalhos têm tamanho fixo e conteúdo simbólico. Eles não seguem o formato real de TCP, IP ou Ethernet, e todas as máscaras de rede são /24.

2. Separação entre as camadas

A separação entre camadas é o eixo do projeto. Ela é garantida pela **estrutura do código**, e não pela disciplina de quem o escreve.

2.1 Interface entre camadas adjacentes

Toda camada é uma subclasse de `Camada` (`camadas.py`) e implementa o mesmo par de métodos:

```
class Camada(ABC):
    numero: int
    nome: str

    def descer(self, pdu: PDU, ctx: Contexto) -> PDU:
        """Recebe a unidade da camada superior, processa e devolve à inferior."""

    def subir(self, pdu: PDU, ctx: Contexto) -> PDU | None:
        """Recebe a unidade da camada inferior, processa e devolve à superior.
        Devolve None quando a unidade é descartada nesta camada."""
```

A interface entre uma camada e a adjacente consiste em duas coisas:

- **a unidade de dados (PDU)**, que uma camada devolve e a pilha entrega à camada vizinha;
- **o contexto (Contexto)**, um objeto criado para cada operação que carrega apenas os parâmetros que uma camada deixa para a adjacente ler. A camada 3, por exemplo, grava nele o vizinho e a interface de

saída, que a camada 2 lê para montar o quadro.

Há uma única exceção à assinatura comum. `CamadaTransporte.descer` devolve uma **lista** de PDUs, uma por segmento, porque a segmentação transforma uma mensagem em várias unidades independentes.

`CamadaRede` tem ainda o método `roteia()`, usado tanto na origem quanto nos roteadores intermediários.

2.2 Como se garante que nenhuma camada acesse uma camada não adjacente

1. **Nenhuma camada guarda referência a outra.** As classes de camada não têm atributo que aponte para a camada de cima ou de baixo. Quem encadeia as chamadas é a **pilha do dispositivo** (`dispositivos.py`): ela chama `17.descer`, entrega o resultado a `16.descer`, e assim por diante. Uma camada não tem como chamar outra, adjacente ou não (figura 1).
2. **Cada cabeçalho carrega o número da camada que o inseriu.** Todo bloco de cabeçalho ou finalizador tem o campo `camada`. A remoção, na subida, passa obrigatoriamente por `PDU.remover_cabecalho(camada)` ou por `conferir_camada()`, que levanta `CabecalhoAlheio` se o número não for o da camada que está removendo. Assim, uma camada só desencapsula o que a sua par (a mesma camada no dispositivo oposto) encapsulou.
3. **A camada 2 não tem acesso à tabela de encaminhamento.** O vizinho e a interface de saída chegam à camada 2 como dados do contexto, já decididos pela camada 3. A classe `CamadaEnlace` não tem método de roteamento, e a ação `ROTEIA` não pertence ao vocabulário da camada 2 (`constantes.py`).
4. **A camada 2 não escreve no pacote.** O par de endereços lógicos é gravado no cabeçalho H3 uma única vez, pela camada 3 da origem. A camada 2 apenas envolve o pacote com H2 e T2, e não tem como alterar o H3.
5. **O quadro nunca é reescrito.** Ao subir, a camada 2 chama `Quadro.consumir()`, que torna o objeto inerte: qualquer leitura posterior levanta `QuadroConsumido`. Na descida, a única forma de obter um quadro é construir um `Quadro` novo, que recebe um identificador novo do contador global (Q1, Q2, ...). Não existe parâmetro para reaproveitar um identificador.

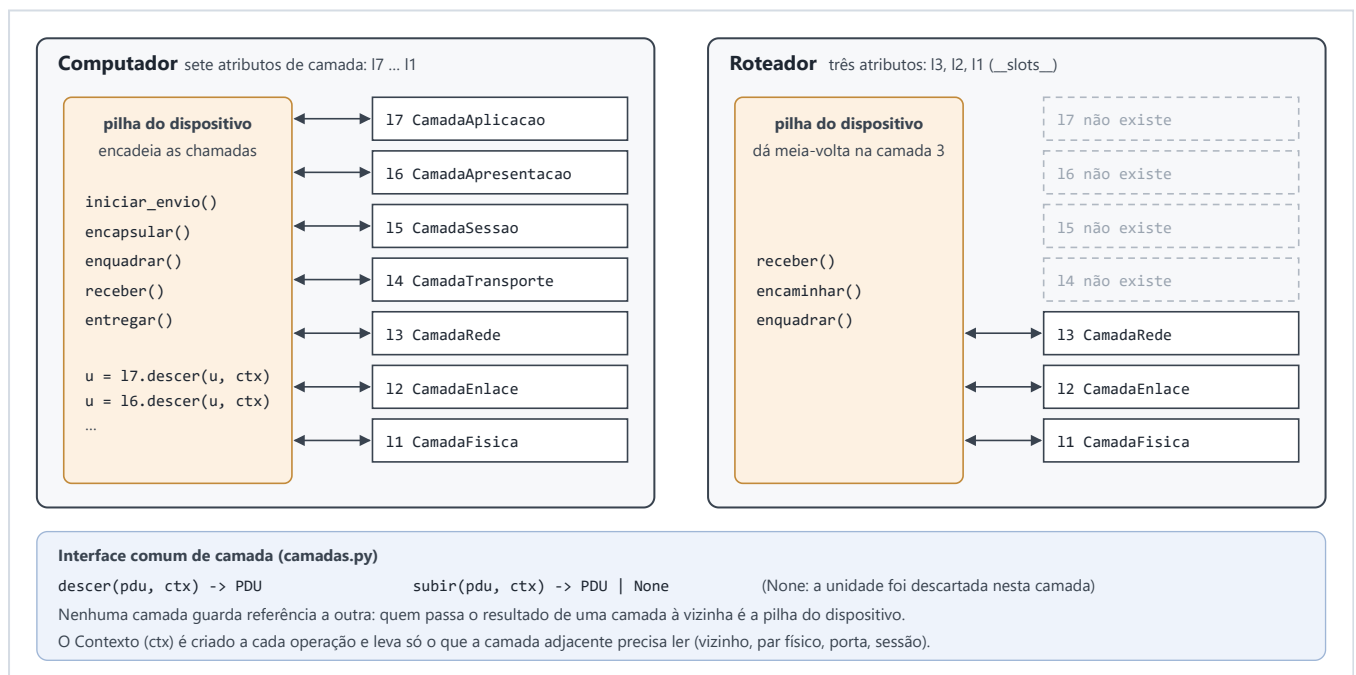


Figura 1: a pilha do dispositivo encadeia as camadas; o roteador não tem as camadas 4 a 7.

2.3 Por que o roteador não consegue ler uma porta

A porta de origem e a de destino ficam no cabeçalho **H4**, inserido pela camada 4 da origem e guardado dentro do pacote. Para ler uma porta, um dispositivo precisaria:

- de um objeto `CamadaTransporte`, a única classe que interpreta o conteúdo do H4; e
- de chegar ao H4, o que exige remover antes o H3, algo que só `CamadaRede.subir()` faz, conferindo o número da camada.

O roteador não atende a nenhuma das duas condições:

- `Roteador.__init__` (por meio de `No.__init__`) instancia apenas 11, 12 e 13. Os objetos das camadas 4 a 7 **não existem** no roteador. Não é que fiquem sem uso: eles nunca são criados.
- As classes `No` e `Roteador` declaram `__slots__` sem `__dict__`. Por isso, uma tentativa de acrescentar em tempo de execução um atributo que não está no molde da classe, como `roteador.14 = CamadaTransporte()`, levanta `AttributeError`.
- O roteador nunca chama `CamadaRede.subir()`. Ele usa `roteia()`, que só lê o endereço lógico de destino do H3 e decide o próximo salto, sem desencapsular. O pacote atravessa o roteador intacto, com o H4 dentro, sem que nenhum código do roteador o interprete.
- Cada operação cria o seu próprio `Contexto`. O roteador nunca vê o contexto da origem, onde a porta foi preenchida.

O resultado é conferido sobre o registro pronto: nenhum evento de roteador tem os campos `porta`, `sessao`, `segmento` ou `processo` preenchidos (teste T-R1).

2.4 As cinco restrições estruturais

#	Restrição	Onde é garantida	Teste
R1	O roteador não lê as camadas 4 a 7	<code>No / Roteador</code> com <code>__slots__</code> e só 11, 12, 13 (<code>dispositivos.py</code>)	<code>test_r1.py</code>
R2	O quadro nunca é reescrito	<code>Quadro.consumir()</code> e contador global de identificadores (<code>pdu.py</code>)	<code>test_r2.py</code>
R3	O endereço lógico é fixo desde a origem; o físico é local a cada salto	O H3 é escrito só em <code>CamadaRede.descer</code> ; o H2 é criado a cada <code>ENQUADRA</code>	<code>test_r3.py</code>
R4	O enlace não escolhe a rota	<code>CamadaEnlace</code> sem <code>roteia()</code> e sem acesso à tabela; o vizinho chega pelo contexto	<code>test_r4.py</code>
R5	Cada camada conversa só com a sua par	<code>Bloco.camada</code> e <code>conferir_camada()</code> (<code>pdu.py</code>)	<code>test_r5.py</code>

3. Diagrama de módulos

3.1 Componentes e dependências

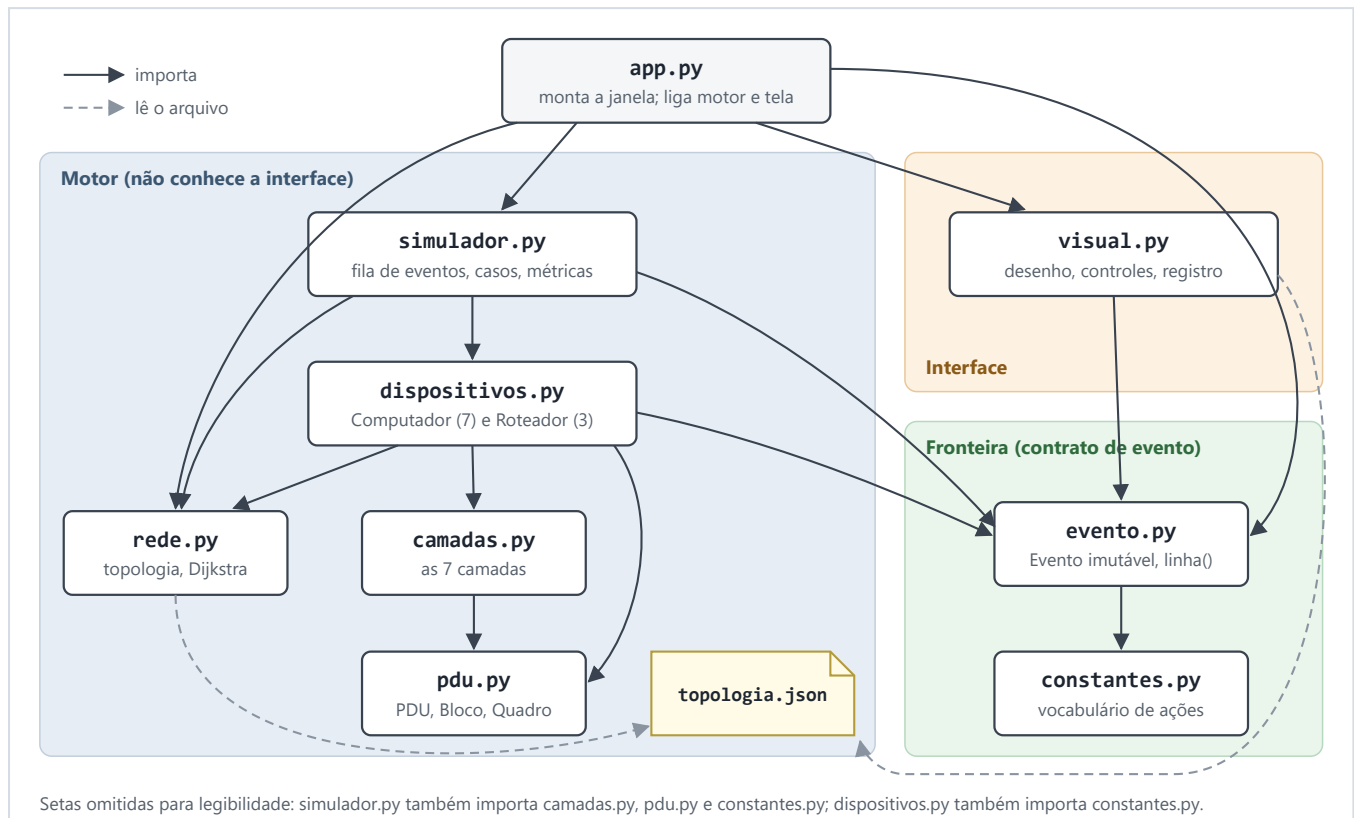


Figura 2: módulos do programa e dependências de importação.

A regra central do diagrama é que **visual.py** **importa apenas evento.py**. A interface nunca chama uma camada, um dispositivo ou o roteador para desenhar a tela: tudo o que ela mostra está nos campos do **Evento**. **app.py** é o único módulo que conhece os dois lados. Ele carrega a topologia, pede ao motor a lista de eventos e a entrega à janela. As regras de importação são verificadas automaticamente pelo teste **test_acoplamento.py**.

3.2 Fluxo de dados, da geração à entrega

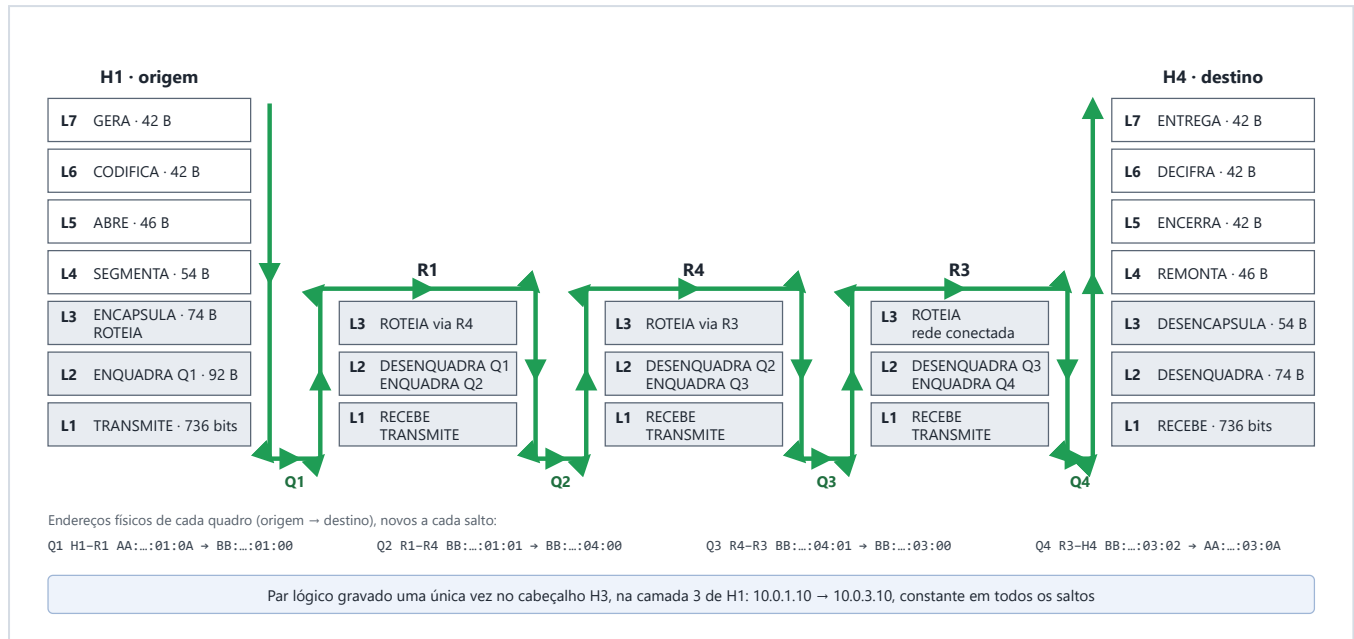


Figura 3: percurso da mensagem no caso C2, de H1 a H4.

1. `app.py` lê o `topologia.json` com `rede.carregar_topologia()`, que valida o arquivo e devolve um objeto `Topologia`.
2. `simulador.executar()` monta um objeto de pilha por dispositivo (`dispositivos.montar_dispositivos()`). A tabela de encaminhamento de cada roteador é derivada do cálculo de menor caminho de `rede.py`.
3. O computador de origem desce as camadas 7 a 4 (`iniciar_envio`) e, para cada segmento, a camada 3 (`encapsular`).
4. A cada salto, o remetente desce as camadas 2 e 1 (`enquadrar`), e cada estação do enlace sobe as camadas 1 e 2 (`receber`). Um roteador decide o próximo salto (`encaminhar`) e repete o salto.
5. O computador de destino sobe as camadas 3 a 7 (`entregar`) e entrega a mensagem ao processo.
6. Cada ação de camada vira um `Evento` na fila do motor (`FilaEventos`). No fim, o evento `METRICAS` é acrescentado e a fila é drenada numa tupla imutável.
7. `visual.py` recebe essa tupla e navega nela por índice.

4. Estrutura da unidade de dados

A unidade de dados é representada por três classes de `pdu.py`. O conteúdo dos cabeçalhos é textual e simbólico; só o bloco de dados carrega octetos reais.

PDU: a unidade de dados de uma camada

Campo	Tipo	Função
<code>nome</code>	texto	Tipo da unidade: "Mensagem", "Segmento", "Pacote" ou "Quadro"
<code>blocos</code>	lista de <code>Bloco</code>	Cabeçalhos já acrescentados, bloco de dados e finalizador, na ordem em que aparecem no fio
<code>tamanho()</code>	método → inteiro	Soma dos tamanhos declarados dos blocos, em octetos

Bloco : cada parte da unidade

Campo	Tipo	Função
rotulo	texto	Nome do bloco: H5, H4, H3, H2 (cabeçalhos), Dados ou T2 (finalizador)
tam	inteiro	Tamanho em octetos. É fixo por convenção para cabeçalhos e igual ao comprimento real para os dados
tipo	texto	"cabeçalho", "dados" ou "finalizador"
conteudo	texto ou bytes	Conteúdo transportado (ver tabela abaixo)
camada	inteiro ou nulo	Número da camada que inseriu o bloco (restrição R5); nulo no bloco de dados

Quadro : subclasse de PDU usada na camada 2

Campo	Tipo	Função
id	texto	Identificador global do quadro (Q1, Q2, ...), obtido do contador no momento da construção
consumir()	método	Destrói o quadro na subida; depois disso, ler blocos levanta QuadroConsumido
copia_recebida()	método	Cópia com o mesmo id, para as outras estações de um segmento de difusão
com_bit_invertido(bit)	método	Mesmo quadro com um bit dos dados invertido (erro de transmissão)

Onde fica cada informação

Informação	Onde é guardada	Formato	Tipo
Dados	Bloco Dados, campo conteudo	Texto na camada 7; octetos cifrados abaixo da camada 6	texto / bytes
Sessão	Cabeçalho H5	S-0001	texto
Portas e segmento	Cabeçalho H4	porta_origem:porta_destino:n:total, por exemplo 5210:443:1:1	texto
Endereços lógicos	Cabeçalho H3	origem:destino, por exemplo 10.0.1.10:10.0.3.10	texto
Endereços físicos	Cabeçalho H2	destino;origem, com o destino antes da origem, como no quadro Ethernet	texto
Verificação de erro	Finalizador T2	CRC-32 em hexadecimal, 8 dígitos	texto

O Evento (evento.py)

O Evento é a fotografia congelada de uma ação. Os campos principais são:

Campo	Tipo	Função
passo	inteiro	Número sequencial no registro, a partir de 1
dispositivo	texto	Quem agiu (H1, R1, ...; -- nos eventos de sistema)
camada	inteiro	1 a 7; 0 nos eventos de sistema

Campo	Tipo	Função
acao	texto	Ação, do vocabulário fechado de constantes.py
descricao	texto	Descrição legível da ação
tamanho	inteiro ou nulo	Tamanho da unidade após a ação, em octetos
pdu	PduInfo	Nome e blocos (rótulo, tamanho, tipo) da unidade, para o desenho
logico	Logico	Par de endereços lógicos de origem e destino
fisico	Fisico	Par de endereços físicos do salto corrente
porta	Porta	Par de portas de origem e destino (só em computadores)
sessao, segmento, processo	vários	Contexto das camadas superiores (só em computadores)
enlace, quadro, bits	vários	Enlace atravessado, identificador do quadro e tamanho em bits
caminho	tupla de textos	Enlaces já percorridos, para o destaque do mapa
estado	texto	"ok", "descartado" ou "erro"
secundario	booleano	Verdadeiro nos descartes por endereço, ocultos por padrão
metricas	Metricas	Só no evento final: eficiência, sobrecarga e totais

O método `Evento.linha()` produz a linha do registro no formato `passo | dispositivo | camada | ação | descrição`, com o tamanho alinhado na coluna 77.

5. Interface dos módulos

5.1 pdu.py

Função ou classe	Recebe	Devolve	O que faz
<code>PDU.remover_cabecalho(camada)</code>	número da camada que remove	(bloco, PDU nova)	Tira o bloco do topo, conferindo que foi inserido pela camada par; senão levanta <code>CabecalhoAlheio</code>
<code>conferir_camada(bloco, camada)</code>	bloco e número da camada	nada	Levanta <code>CabecalhoAlheio</code> se o bloco não pertence à camada
<code>Quadro(blocos)</code>	lista de blocos	<code>Quadro</code>	Constrói um quadro com identificador novo
<code>reiniciar_contador_quadros()</code>	nada	nada	Zera a numeração de quadros antes de cada caso

5.2 camadas.py

Classe	descer(pdu, ctx)	subir(pdu, ctx)
<code>CamadaAplicacao</code> (7)	GERA: cria a <code>Mensagem</code> a partir de <code>ctx.mensagem</code>	ENTREGA: devolve a mensagem sem alteração
<code>CamadaApresentacao</code> (6)	CODIFICA: converte para UTF-8 e cifra com XOR (chave <code>REDES</code>)	DECIFRA: aplica o mesmo XOR e volta ao texto
<code>CamadaSessao</code> (5)	ABRE: gera o identificador <code>S-NNNN</code> e insere o H5 (+4 B)	ENCERRA: remove o H5 conferido e grava a sessão em <code>ctx</code>

Classe	descer(pdu, ctx)	subir(pdu, ctx)
CamadaTransporte (4)	SEGMENTA : divide os dados em segmentos de até 64 B e insere o H4 (+8 B) em cada um; devolve uma lista	DEMULTIPLEXA/REMONTA : remove o H4, guarda o pedaço por porta de origem e devolve a mensagem só quando todos os segmentos chegaram (senão, <code>None</code>)
CamadaRede (3)	ENCAPSULA : insere o H3 (+20 B) com o par lógico	DESENCAPSULA : remove o H3 no destino final
CamadaRede.rroteia(pdu, ctx)	consulta <code>ctx.tabela_encaminhamento</code> pelo prefixo mais longo	ROTEIA : grava vizinho e interface em <code>ctx</code> ; DESCARTA (<code>None</code>) se não há rota
CamadaEnlace (2)	ENQUADRA : constrói um <code>Quadro</code> novo com H2 (+14 B) e T2 (+4 B) contendo o CRC-32	DESENQUADRA/DESCARTA/IGNORA : confere o CRC e o endereço físico, consome o quadro e devolve o pacote ou <code>None</code>
CamadaFisica (1)	TRANSMITE : calcula <code>bits = octetos × 8</code> ; não altera a unidade	RECEBE : idem, na outra ponta

Funções auxiliares: `crc32(dados)` (CRC-32 com tabela própria, polinômio 0xEDB88320), `par_logico_do_h3(texto)`, `par_fisico_do_h2(texto)` e `reiniciar_contador_sessoes()`.

5.3 dispositivos.py

Função ou classe	Recebe	Devolve	O que faz
No	dispositivo e topologia	objeto de pilha	Metade baixa comum: 11, 12, 13 e a tabela de encaminhamento
No.enquadrar(pacote, vizinho, interface_saida)	pacote e decisão da camada 3	Transmissao	ENQUADRA + TRANSMITE: resolve os endereços físicos do salto e cria as entregas a cada estação do enlace
No.receber(entrega)	uma entrega do enlace	Recepcao	RECEBE + DESENQUADRA, DESCARTA ou IGNORA
Roteador.encaminhar(pacote)	pacote em trânsito	Encaminhamento	ROTEIA ou DESCARTA por falta de rota
Computador.iniciar_envio(fluxo)	fluxo do caso	Envio	Descida das camadas 7 a 4 (GERA ... SEGMENTA)
Computador.encapsular(envio, segmento)	um segmento	Encaminhamento	ENCAPSULA + ROTEIA na origem
Computador.entregar(pacote, processo, ...)	pacote no destino	ações de camada	Subida das camadas 3 a 7 (DESENCAPSULA ... ENTREGA)
montar_dispositivos(topologia)	topologia	{nome: No}	Cria um Computador ou Roteador por dispositivo

Os métodos devolvem objetos `AcaoCamada`: o retrato do evento sem `passo`, `fluxo` e `descricao`, que são preenchidos pelo motor.

5.4 rede.py

Função ou classe	Recebe	Devolve	O que faz
<code>carregar_topologia(caminho)</code>	caminho do arquivo (padrão: ao lado do programa)	<code>Topologia</code>	Lê, interpreta e valida o JSON; qualquer problema vira <code>ErroTopologia</code> com mensagem legível
<code>validar_topologia(topologia)</code>	topologia	nada	Aplica as verificações V-01 a V-10 (nomes e endereços únicos, grafo conexo, gateway na rede local, limite de segmento etc.)
<code>pasta_base()</code>	nada	<code>Path</code>	Pasta do executável (quando empacotado) ou do código-fonte; única resolução de caminho do programa
<code>menor_caminho(origem, arestas, ...)</code>	vértice de origem e arestas	<code>ArvoreCaminhos</code>	Cálculo de menor custo a partir de um dispositivo, com desempate determinístico
<code>Topologia.tabela_encaminhamento(roteador)</code>	nome do roteador	<code>TabelaEncaminhamento</code>	Uma rota por rede alcançável: prefixo, próximo salto, interface e custo
<code>Topologia.com_enlace_ativo(id, ativo=...)</code>	enlace e estado	<code>Topologia</code> nova	Cópia da topologia com o enlace ligado ou desligado; base do recálculo de rota
<code>Topologia.caso(id)</code>	identificador	<code>Caso</code>	Um caso declarado no arquivo, com os fluxos já resolvidos

5.5 simulador.py

Função ou classe	Recebe	Devolve	O que faz
<code>executar(topologia, caso)</code>	topologia e caso	tupla de <code>Evento</code>	Executa o caso inteiro e devolve o registro numerado
<code>executar_caso(topologia, id)</code>	topologia e identificador	tupla de <code>Evento</code>	Atalho para <code>executar</code> a partir do identificador
<code>com_intervencao(caso, evento_externo)</code>	caso e falha	<code>Caso</code> novo	Acrescenta uma queda de enlace ou um erro de bit ao caso

Função ou classe	Recebe	Devolve	O que faz
<code>FilaEventos</code>	eventos agendados por instante	tupla de <code>Evento</code>	Fila de prioridade por (<code>instante</code> , <code>ordem de inserção</code>) que atribui os números de passo
<code>formatar_registro(topologia, caso, eventos)</code>	eventos	texto	Cabeçalho mais uma linha por evento
<code>principal(argv)</code>	argumentos de linha de comando	código de saída	Modo textual: <code>python simulador.py --caso C2 --log saida.txt</code>

5.6 `visual.py` e `app.py`

Função ou classe	Recebe	Devolve	O que faz
<code>Navegador(eventos)</code>	tupla de eventos	objeto	Índice sobre a lista: <code>proximo()</code> , <code>anterior()</code> , <code>ir_para(i)</code> , <code>ate_agora()</code>
<code>Janela(...)</code>	planta, navegador, executor do caso e parâmetros	objeto	A tela inteira; <code>montar(raiz)</code> cria os componentes, e <code>redesenhar()</code> desenha o estado do passo corrente
<code>carregar_planta(caminho)</code>	caminho do <code>topologia.json</code>	<code>Planta</code>	Lê as posições dos dispositivos para o mapa (sem usar <code>rede.py</code>)
<code>MapaDaRede</code> , <code>PilhasDosDispositivos</code> , <code>UnidadeDeDados</code> , <code>PaineisDeEndereco</code> , <code>RegistroNaTela</code>	estado derivado dos eventos	nada	Cada região da tela
<code>app.principal(argv)</code>	<code>--caso</code> , <code>--topologia</code>	código de saída	Carrega a topologia, executa o caso inicial e abre a janela; erros viram caixa de mensagem
<code>app.montar_executor(topologia)</code>	topologia	função	Função que a janela chama para reexecutar um caso com intervenções

6. Parâmetros configuráveis

Parâmetro	Onde se altera	Valor padrão	Faixa válida
Tamanho dos cabeçalhos: L7, L6, L5, L4, L3, L2 e T2	Constantes <code>_TAM_CABECALHO_L5</code> , <code>_TAM_CABECALHO_L4</code> , <code>_TAM_CABECALHO_L3</code> , <code>_TAM_CABECALHO_L2</code> e <code>_TAM_FINALIZADOR_L2</code> em <code>camadas.py</code>	0, 0, 4, 8, 20, 14 e 4 octetos	Inteiros ≥ 0

Parâmetro	Onde se altera	Valor padrão	Faixa válida
Limite de segmentação da camada 4	"parametros" → "limite_segmento" no <code>topologia.json</code>	64 octetos de carga útil	Inteiro positivo, no mínimo o tamanho da maior mensagem não segmentada mais 4 (o H5); na topologia de referência, ≥ 46
Caminho do arquivo de topologia	Opção <code>--topologia ARQUIVO</code> do programa ou do modo textual	<code>topologia.json</code> na pasta do executável (<code>pasta_base()</code>)	Qualquer arquivo JSON válido, em UTF-8
Velocidade de reprodução	"parametros" → "velocidades_ms" no <code>topologia.json</code>	lenta = 1200, media = 500, rapida = 150 ms por passo; media selecionada ao abrir	Inteiros positivos, em milissegundos
Caso inicial	Opção <code>--caso</code>	Primeiro caso do arquivo	Um identificador existente no arquivo

O bloco "cabecalhos" do `topologia.json` registra os mesmos tamanhos da tabela para quem lê o arquivo. O motor, porém, usa as constantes de `camadas.py`; desse bloco, só o valor de `L5` é usado, na validação do limite de segmentação. Alterar o tamanho de um cabeçalho exige mudar a constante correspondente, e os valores de referência dos testes mudam junto.

A chave da cifra (REDES), o polinômio do CRC-32 (`0xEDB88320`) e o formato da sessão (S- com 4 dígitos) aparecem em "parametros" como documentação e estão fixados em `camadas.py`.

7. Funcionamento interno

7.1 A descida na origem

A camada 7 cria a `Mensagem` com o texto do caso (42 octetos em C2). A camada 6 converte para UTF-8 e cifra com XOR, sem alterar o comprimento. A camada 5 gera a sessão e insere o H5 (46 B). A camada 4 divide a carga em segmentos de até 64 octetos e insere o H4 em cada um (54 B). Para cada segmento, a camada 3 insere o H3 com o par lógico de ponta a ponta (74 B) e consulta a tabela de encaminhamento.

A tabela do computador tem duas entradas: a própria rede /24, com entrega direta, e `0.0.0.0/0` apontando para o gateway. A consulta por prefixo mais longo reproduz a decisão "mesma rede → entrega direta; outra rede → gateway" sem código especial. A camada 2 constrói o quadro Q1 com H2 e T2 (92 B), e a camada 1 o transmite (736 bits).

7.2 O salto: o quadro é desmontado e outro é construído

Ao receber, a camada 1 entrega o quadro à camada 2. A camada 2 recalcula o CRC-32 sobre tudo o que precede o finalizador e compara com o valor do T2. Depois, confere se o endereço físico de destino é o da interface que recebeu. Em seguida, **consome** o quadro e entrega o pacote (74 B) à camada 3. O quadro recebido deixa de existir.

No roteador, a camada 3 executa `roteia()`: lê o destino lógico do H3, escolhe a rota e grava o vizinho e a interface de saída no contexto. A camada 2 constrói então um **quadro novo**, com identificador novo e outro par de endereços físicos: o da interface de saída e o do vizinho naquele enlace. Em C2, isso produz Q1, Q2, Q3 e Q4, um por enlace, cada um com seu par físico, enquanto o par lógico permanece o mesmo.

Num segmento de difusão (as redes A, B e C), o quadro chega a todas as estações. Cada uma recebe uma cópia com o mesmo identificador. As que não são o destino físico registram `IGNORA`, um evento secundário que fica oculto por padrão.

7.3 A tabela de encaminhamento

A tabela de cada roteador é derivada do cálculo de menor caminho sobre o grafo da topologia. Os vértices são os dispositivos, e as arestas são os enlaces com os custos do arquivo. Um segmento de difusão vira arestas de custo 0 entre todas as suas pontas, e computadores não servem de trânsito. O cálculo é um relaxamento até ponto fixo, que produz o mesmo resultado do algoritmo de Dijkstra e trata sem casos especiais as arestas de custo 0.

Cada linha da tabela contém o prefixo da rede de destino, o próximo salto, a interface de saída e o custo. A consulta escolhe a entrada de prefixo mais longo que contém o destino. Como todas as máscaras são /24, isso equivale a comparar os três primeiros octetos. Sem entrada compatível, a camada 3 emite `DESCARTA`, o pacote é destruído e nenhuma camada é acionada depois (caso C5).

Quando um enlace é derrubado (caso C4 ou botão **Derrubar enlace**), o motor cria uma cópia da topologia com o enlace inativo **antes** de montar os dispositivos. As tabelas nascem já recalculadas: R1 passa a alcançar a Rede C via R2, com custo 3. O registro começa com o evento `ENLACE_FORA`.

7.4 A remontagem no destino

No destino, a camada 3 remove o H3 e entrega o segmento à camada 4. A camada 4 remove o H4 e guarda os octetos do segmento num espaço separado **por porta de origem**. É isso que separa as duas conversas do caso C3, que chegam ao mesmo destino e à mesma porta. Enquanto faltar algum segmento, `subir()` devolve `None` e nada sobe. Quando o último chega, os pedaços são concatenados na ordem da numeração e a mensagem remontada segue para as camadas 5, 6 e 7. Em C7, os três segmentos (64, 64 e 56 octetos de carga) viajam um após o outro pela rota inteira, e a remontagem só acontece depois do terceiro.

7.5 Quando um quadro chega corrompido

O erro de transmissão é injetado pelo motor no quadro **em trânsito**: entre o `TRANSMITE` do remetente e a recepção na outra ponta, um bit do bloco de dados é invertido (evento `CORROMPE`). A camada 2 de quem recebe recalcula o CRC-32, encontra divergência com o T2 e registra `DESCARTA` com estado `erro`. O quadro é consumido e a camada 3 não é acionada. Não há retransmissão: o fluxo termina ali, a mensagem não é entregue e a eficiência do caso é 0%.

7.6 O motor de eventos

`simulador.py` percorre cada fluxo e agenda cada ação numa fila de prioridade ordenada por (`instante`, `ordem de inserção`). O instante é um tempo lógico, que avança um a cada salto e não tem unidade. Em fluxos concorrentes (C3), cada fluxo recomeça no instante 0, e a drenagem da fila intercala os dois salto a salto. No fim, o evento `METRICAS` é calculado a partir dos próprios eventos registrados:

- **dados**: octetos da mensagem, contados uma vez, e só se ela foi entregue;
- **transmitido**: soma dos tamanhos de todos os quadros transmitidos em todos os enlaces;
- $\eta = \text{dados} / \text{transmitido}$ e **sobrecarga** = $1 - \eta$.

8. Convenções de simulação

Estas decisões determinam os números produzidos. Com outras convenções, a mesma mensagem geraria resultados diferentes.

Convenção	Decisão adotada
Tamanho dos cabeçalhos	L7 e L6: 0 (produzem metadado, não cabeçalho). L5: 4. L4: 8. L3: 20. L2: 14 de cabeçalho e 4 de finalizador. L1: 0; apenas converte octetos em bits ($\times 8$)
Conteúdo dos cabeçalhos	Simbólico e de tamanho fixo; não segue o formato real de TCP, IP ou Ethernet
Tamanho da mensagem	Número de octetos do texto codificado em UTF-8
Cifra da camada 6	XOR com a chave REDES repetida, que preserva o comprimento exato
Limite por segmento	64 octetos de carga útil por segmento, sem contar o H4. Diverge da convenção C2 da especificação, que fixa 40; ver 8.1
Cabeçalho de sessão e segmentação	O H5 é inserido uma vez só , antes da segmentação, e viaja apenas no primeiro segmento , ocupando 4 dos 64 octetos dele. Em C7: $4 + 180 = 184$ octetos viram segmentos de 64 (4 de H5 + 60 de dados), 64 e 56
Ordem dos segmentos	Cada segmento percorre a rota inteira antes do próximo partir
Desempate entre caminhos de mesmo custo	Vence o caminho cujo próximo salto tem o menor nome em ordem lexicográfica (R1 antes de R2). É a única regra de desempate
Custo dos segmentos de difusão	0: atravessar uma rede local não soma custo
Numeração dos quadros	Contador global e contínuo por execução (Q1, Q2, ...), reiniciado a cada caso e nunca por dispositivo. Cada salto cria um quadro novo com o próximo número. As cópias de um mesmo quadro num segmento de difusão compartilham o identificador
Numeração dos passos	Global e contínua a partir de 001. Eventos secundários (IGNORA) repetem o número do último evento principal, para que o registro padrão não tenha lacunas
Numeração das sessões	Contador global S-0001, S-0002, ..., reiniciado a cada caso
Quadro descartado por erro	É destruído na camada 2 de quem recebeu; não há retransmissão; o fluxo termina e a mensagem conta como não entregue ($\eta = 0\%$). Os quadros já transmitidos, inclusive o corrompido, entram no total transmitido
Verificação de erro	CRC-32, polinômio 0xEDB88320, calculado sobre o H2, o pacote e os rótulos e tamanhos dos blocos; o erro de bit é aplicado apenas ao bloco de dados
Eficiência	$\eta = \text{octetos de dados entregues} \div \text{octetos transmitidos em todos os enlaces}$

Valores resultantes na topologia de referência:

Caso	Dados	Enlaces	Transmitido	η
C1	42 B	1	92 B	45,7%
C2	42 B	4	368 B	11,4%
C3	84 B	4 + 4	736 B	11,4%
C4	42 B	4	368 B	11,4%
C5	42 B	1	92 B	0% (não entregue)
C6	42 B	3	276 B	0% (não entregue)
C7 +	180 B	4 $\times$ 3	1288 B	14,0%

† Diverge da especificação, que registra 968 B e $\eta = 10,3\%$ para E7. Ver 8.1.

8.1 Divergência conhecida: a convenção C2 e os cenários E1–E6

A tabela acima reproduz os valores da especificação em C1 a C6 e diverge dela em C7, que o enunciado registra como 968 octetos transmitidos e $\eta = 10,3\%$. A divergência é deliberada: as convenções da especificação são inconsistentes entre si, e nenhuma implementação pode satisfazer todas ao mesmo tempo.

O conflito. A convenção C2 fixa em 40 octetos o máximo que a camada 4 pode colocar num segmento, contando o cabeçalho de sessão, que a camada 5 insere antes. Os cenários E1 a E6 usam a mensagem de referência de 42 octetos, que chega à camada 4 com 46 — acima do limite. Pela C2, esses seis cenários deveriam produzir dois segmentos e, portanto, o dobro dos quadros. A própria especificação, porém, apresenta E1 com um único quadro de 92 octetos e E2 com quatro quadros e 368 octetos, valores que só existem com segmento único.

Limite adotado	E1 a E6	E7
40 (convenção C2)	errados: dois segmentos onde o enunciado mostra um	exato: 968 B, $\eta = 10,3\%$
46 ou mais	exatos	totais diferentes dos do enunciado

Não há valor que atenda aos dois lados.

A decisão. O simulador adota 64 octetos (`limite_segmento`), preservando os seis cenários cujos valores a especificação apresenta de forma consistente. Como 104 octetos cabem em dois segmentos de 64, e E7 existe para demonstrar segmentação em três partes com remontagem única, a mensagem de C7 foi ajustada para 180 octetos. C7 mantém assim tudo o que o cenário se propõe a verificar — três segmentos numerados, doze quadros em sequência contínua, travessia sequencial e uma única remontagem no destino — e difere apenas nos totais.

O modelo reproduz E7. A divergência está nas convenções, não no motor. Com `limite_segmento` igual a 40 e a mensagem de 100 octetos do enunciado, o simulador produz exatamente os valores da especificação: três segmentos de 40, 40 e 24 octetos de carga, doze quadros, 968 octetos transmitidos e $\eta = 10,3\%$. O que não é possível é obter esses valores no mesmo arquivo de topologia que atende E1 a E6.

Onde isso é verificado. A verificação V-10, em `rede.py`, recusa qualquer `limite_segmento` que fatiaria a mensagem de referência, para que uma alteração desse parâmetro não mude os registros de C1 a C6 em silêncio.

9. Dependências e ambiente

Item	Valor
Linguagem	Python 3.11 ou mais recente
Bibliotecas	Somente a biblioteca padrão do Python: <code>tkinter</code> / <code>ttk</code> (interface), <code>json</code> , <code>dataclasses</code> , <code>ipaddress</code> , <code>itertools</code> , <code>argparse</code> , <code>pathlib</code> e <code>unittest</code> (testes). Nenhuma dependência externa
Testes	<code>python -m unittest discover -s tests</code> : 692 testes, executáveis sem interface gráfica
Modo textual	<code>python simulador.py --caso C2 --log saida.txt</code>
Interface	<code>python app.py</code> ou <code>python app.py --caso C4</code>
Empacotamento	PyInstaller 6.22.3, sobre Python 3.14 para Windows 64 bits

O executável é gerado a partir de `app.py`, o único módulo que reúne o motor e a interface:

```
pip install pyinstaller
pyinstaller --onefile --windowed --name SimuladorOSI app.py
```

O resultado é `dist/SimuladorOSI.exe`, um arquivo único de cerca de 12 MB que inclui o interpretador Python e o `tkinter`. Ele é copiado para a raiz do repositório, ao lado do `topologia.json`, e versionado junto com o código; a cada mudança no código, o executável é gerado de novo e substituído. A opção `--windowed` suprime o console, e por isso os erros de carga aparecem numa caixa de mensagem.

O `topologia.json` **não** é embutido no executável. Ele fica ao lado do `SimuladorOSI.exe` e é localizado em tempo de execução por `pasta_base()`, que usa a pasta do executável (`sys.executable`) quando o programa está empacotado e a pasta do código-fonte nos demais casos. É isso que permite trocar a rede sem gerar um novo executável. Nenhum caminho absoluto aparece no código.